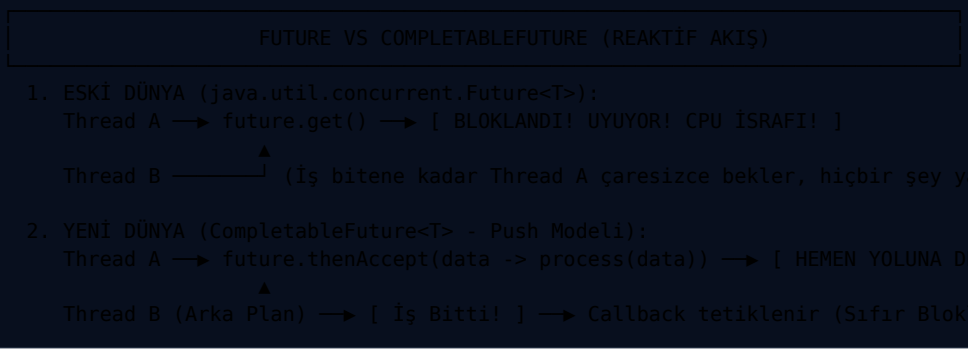
37

BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Future<T> vs CompletableFuture<T>: Polling Kâbusundan Reaktif İtmeye Geçiş

Future<T> vs CompletableFuture<T>: Escaping the Polling Nightmare

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski Future, pizza siparişi verip kapının deliğine gözünü dayayarak 45 dakika boyunca hiçbir yere kımıldamadan kuryeyi beklemeye benzer (`future.get()` işi bitene kadar thread'i kilitler). CompletableFuture ise kapıya 'Pizza gelince zili çal, pizzayı fırına atıp kutuyu çöpe at' yazılı bir akıllı talimat yapıştırmaktır. Kurye geldiğinde talimatlar otomatik yürütülür, sen o sırada salonda televizyon izlemeye devam edersin!

🇬🇧 ENGLISH (ELI5)

The legacy `Future\<T>` is like ordering food and standing motionlessly with your eye glued to the door peephole for 45 minutes (`future.get()` blocks the thread dead). `CompletableFuture\<T>` is leaving an intelligent sticky note on the door: 'When the courier arrives, put pizza on the table, tip $5, and text me'. You continue watching television. When the data is ready, callbacks push reactions forward without blocking any CPU thread!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 5'te gelen `Future\<T>` arayüzünün 3 devasa zafiyeti vardı: 1) Bloklayan Çekme Modeli: Sonucu almak için `future.get()` çağrılır ve çağıran thread uyutulur (Context Switch maliyeti). 2) Zincirleme Yapılamaz: Bir Future'ın çıktısını diğer bir Future'ın girdisi yapacak deklaratif yapı yoktur. 3) Manuel Tamamlama İmkânsızdır: Dışarıdan bir etkinlik (web-socket, JMS mesajı) geldiğinde Future'ı el ile sonuçlandıramazsınız. Java 8 ile gelen `CompletableFuture\<T>`, `CompletionStage` ve `Future` arayüzlerini birleştirir. ForkJoinPool.commonPool() üzerinde veya özel thread havuzlarında asenkron görevleri non-blocking (olay güdümlü) zincirler halinde yürütür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Shortcomings of legacy `Future\<T>`: 1) Blocking Polling Trap: Callers are trapped in synchronous blocking via `future.get()`, squandering expensive kernel threads. 2) Absence of Declarative Chaining: Cannot natively express 'execute task B when task A completes'. 3) Immutable Completion: Callers cannot externally resolve or fail a Future instance. `CompletableFuture\<T>` implements both `Future\<T>` and `CompletionStage\<T>`. It adopts the Promise pattern: tasks push transformations down a directed acyclic graph (DAG) executed on `ForkJoinPool.commonPool()` or custom `Executor` contexts without idling caller threads.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Non-blocking CompletableFuture ile Sipariş İşleme
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executors;

public class NonBlockingOrderPipeline {
    record Order(String id, double amount, String status) {}

    public static void main(String[] args) {
        try (var customPool = Executors.newFixedThreadPool(4)) {
            // Asenkron veri çekme başlat (Non-blocking!)
            CompletableFuture<Order> orderFuture = CompletableFuture.supplyAsync(() -> {
                System.out.println("Sipariş çekiliyor: " + Thread.currentThread());
                return new Order("ORD-9821", 250.0, "PENDING");
            }, customPool);

            // Reaktif zincir: Veri geldiğinde otomatik işlet (Bloklama yok!)
            orderFuture
                .thenApply(order -> new Order(order.id(), order.amount() * 1.20, "TAX_INCLUDED"))
                .thenAccept(processed -> {
                    System.out.println("İşlem Tamamlandı: " + processed + " Thread: " +
Thread.currentThread());
                });

            System.out.println("Ana thread asla bloklanmadı, diğer işlere devam ediyor...");
            Thread.sleep(100); // Demo amaçlı arka plan işinin bitmesini bekle
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Argümansız supplyAsync(Supplier) varsayılan olarak ForkJoinPool.commonPool() kullanır! Ağ/veritabanı gibi bloklayan (I/O-bound) işleri commonPool'a yüklerseniz havuzdaki tüm CPU çekirdeklerini kilitler ve sistemdeki paralel stream'leri felç edersiniz. I/O işleri için daima özel bir Executor (veya Java 21+ Virtual Thread Executor) geçin!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Executing `supplyAsync()` without an explicit executor defaults to `ForkJoinPool.commonPool()`. Flooding the common pool with blocking I/O (database, REST calls) exhausts CPU-worker threads, starving parallel streams and application background tasks. Always supply a dedicated I/O thread pool or a Virtual Thread executor!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Future.get() çağrısı thread'i kilitler. CompletableFuture ise reaktif itme (push) modelidir; iş bittiğinde sonraki aşama otomatik tetiklenir.

🎯 Key Takeaways for the Interview

• `Future.get()` blocks threads synchronously. `CompletableFuture` employs non-blocking event-driven completion stages via push semantics.